Spring Container: BeanFactory & ApplicationContext

Recap: Where We Are So Far

So far, we've understood:

- **Dependency Injection** removes object creation from classes
- **Inversion of Control** shifts responsibility to Spring
- Spring manages **object creation, wiring, and lifecycle**

Now we answer:
How does Spring actually do this?

The answer: **Spring Container**

What Is a Spring Container?

The **Spring Container** is the core engine of the Spring Framework.

It is responsible for:

- Creating beans
- Injecting dependencies
- Managing bean lifecycle
- Destroying beans

You provide:

- Classes (beans)
- Configuration metadata

Spring provides:

- Fully initialized, wired, managed objects

Configuration Metadata — How Spring Knows What to Do

Spring needs instructions to:

- Know which classes to instantiate
- How to inject dependencies

- What scope, lifecycle, values to apply

This instruction is called **Configuration Metadata**.

It can be provided using:

1. XML Configuration
2. Annotations
3. Java-based configuration

For Day 4, we focus on **XML configuration** to understand fundamentals clearly.

Creating the Spring Container

Spring provides **two core container interfaces**:

Container	Purpose
BeanFactory	Simple applications
ApplicationContext	Enterprise applications

These are **interfaces**, so we use their implementation classes.

BeanFactory — Basic Container

When to Use:

- Small applications
- Memory-constrained environments
- Lazy initialization needed

Implementation Class:

XmlBeanFactory (older, but important conceptually)

Step 1: Create XML Configuration (config.xml)

```xml
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
       http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="sim" class="com.example.AirtelSim"/>

    <bean id="mobile" class="com.example.Mobile">
        <property name="sim" ref="sim"/>
    </bean>

</beans>
```

Step 2: Load XML Using Resource Interface

Spring uses Resource abstraction to load configuration.

Implementation classes:

- ClassPathResource
- FileSystemResource

```
Resource resource = new ClassPathResource("config.xml");
BeanFactory factory = new XmlBeanFactory(resource);
Mobile mobile = factory.getBean("mobile", Mobile.class);
```

✓ Beans are created **only when requested** (lazy)

ApplicationContext — Enterprise Container

Why ApplicationContext Is Preferred:

- Eager initialization
- Better performance tuning
- Event handling
- Internationalization (i18n)
- AOP & integration support

Common Implementation Classes:

Class	Use Case
ClassPathXmlApplicationContext	XML inside project
FileSystemXmlApplicationContext	XML outside project
XmlWebApplicationContext	Web apps
AnnotationConfigApplicationContext	Java config
AnnotationConfigWebApplicationContext	Web + Java config

Example: ClassPathXmlApplicationContext

```
ApplicationContext context =
    new ClassPathXmlApplicationContext("config.xml");

Mobile mobile = context.getBean("mobile", Mobile.class);
```

✓ Beans created at startup
✓ Faster runtime access
✓ Production-grade

Key Differences: BeanFactory vs ApplicationContext

Feature	BeanFactory	ApplicationContext
Initialization	Lazy	Eager
Performance	Slower at runtime	Faster
Enterprise features	No	Yes
Recommended	❌	✅

Real projects use ApplicationContext

How Spring Uses IoC & DI Internally

- Spring Container reads configuration
- Creates objects
- Resolves dependencies
- Injects dependencies
- Manages lifecycle

Classes never call new for business components.

DI in Enterprise Layered Architecture

Spring injects dependencies across layers:

- Controller → Service
- Service → Repository
- Repository → DataSource

Each layer:

- Depends on abstraction
- Is independently testable
- Is loosely coupled

Common Interview Questions

1. “What exactly is a Spring Bean?”

Wrong answer (basic):
A Java object managed by Spring.

Correct interview answer:
A Spring Bean is a Java object whose **entire lifecycle (creation, dependency injection, initialization, and destruction)** is managed by the **Spring Container**.

Interviewers expect you to say **“lifecycle”**, not just “object”.

2.“What does the Spring Container really do?”

Most candidates say:

It creates objects and injects dependencies.

What interviewers expect:

- Reads configuration metadata
- Instantiates beans
- Resolves dependency graph
- Applies post-processors
- Manages full lifecycle
- Handles scope, proxying, and destruction

💡 **Keyword to drop:**

“Spring Container is a lifecycle and dependency orchestration engine.”

3.**“DI vs IoC — what’s the real difference?”**

This is a **classic trap**.

Correct framing:

- **IoC** → Principle (design philosophy)
- **DI** → Implementation (technique)

💡 Interview one-liner:

“IoC defines *who controls*, DI defines *how dependencies are supplied*.”

4.**“Why is tight coupling bad even with ONE implementation?”**

This question filters juniors from seniors.

Expected reasoning:

- Tight coupling is about **control**, not count
- new inside a class:
    - breaks SRP
    - blocks testability
    - forces recompilation on change
    - prevents mocking
- DI future-proofs architecture

💡 Killer line:

“Spring solves tomorrow’s change, not today’s simplicity.”

5. **“Why does Spring prefer constructor injection?”**

Correct senior-level reasons:

- Ensures mandatory dependencies
- Makes object immutable
- Enables fail-fast behavior
- Simplifies testing
- Prevents partially constructed objects

🚫 Wrong answers:

- “Because Spring recommends it”
- “Because it’s cleaner”

6.**“What happens if I don’t use DI?”**

Interviewers expect **consequences**, not ideology.

Correct points:

- Tight coupling
- Hard-to-test code
- Hidden dependencies
- Poor scalability
- Fragile architecture

7. **“Who creates objects in Spring?”**

This question leads directly to IoC.

Correct answer:

The Spring Container creates, configures, and manages objects based on configuration metadata.

If candidate says:

“Spring creates objects using new”

❌ Immediate red flag.

8.**“What is configuration metadata?”**

Most people fail here.

Correct answer:

Configuration metadata is **instructional data** that tells Spring:

- what to instantiate
- how to wire
- when to initialize
- how to destroy

Formats:

- XML

- Annotations
- Java Configuration

9.**“Is Spring Container the same as JVM?”**

Very common trick question.

Correct answer:

- JVM manages memory
- Spring Container manages **objects**
- Spring runs *inside* JVM

💡 Say:

“JVM manages memory, Spring manages relationships.”

10. **“Why ApplicationContext over BeanFactory?”**

Interviewers don’t want the table — they want **reasoning**.

Correct explanation:

- ApplicationContext eagerly loads beans
- Faster runtime access
- Better enterprise support
- AOP, events, i18n, lifecycle callbacks

💡 Line to remember:

“BeanFactory is minimalistic; ApplicationContext is production-grade.”

11.**“Is BeanFactory used in real projects?”**

Correct answer:

Rarely. ApplicationContext is the standard in enterprise applications.

But also add:

“BeanFactory concepts still matter for understanding Spring internals.”

12.**“What is the Spring Container internally?”**

Expected:

- Interface-based design
- Uses factory pattern
- Uses reflection
- Uses proxying (later topics)
- Maintains dependency graph

13.**“Can Spring manage non-Spring classes?”**

Correct answer:

Yes, as long as Spring is instructed via configuration metadata.

14. **“Does Spring Boot remove IoC or DI?”**

Common confusion.

Correct answer:

No. Spring Boot **builds on Spring Core**.
IoC, DI, and Container remain exactly the same.

💡 Add:

“Spring Boot simplifies configuration, not architecture.”

15. **“What is the role of the container in testing?”**

Strong interview angle.

Expected:

- Mock injection
- Test-specific configuration
- Context isolation
- Faster feedback

16. **“What is NOT a Spring-managed object?”**

Good trick question.

Answer:

- Objects created using new outside container
- Utility/value objects
- Simple immutable objects (BigDecimal, LocalDate)

17. **When is new acceptable?**

Interviewers love this nuance.

Correct answer:
Use new when:

- Object has no behavior
- No external dependency
- Immutable
- No lifecycle concern

Examples:

new BigDecimal("100");

new LocalDate();

Not acceptable for:

- Services
- Repositories
- Integrations
- Business components

18. **One-Line Summary (MEMORIZE)**

“Spring Core is about removing object creation, inverting control, and letting a container manage lifecycle and wiring for scalable, testable enterprise applications.”